

**Choosing the Right Model for Each Task: Response Time, Compute Cost, and Routing Across Four
Vision-Language Models**

Olena McMurtrey

Probability and Statistics with Python

October 9, 2026

Author Note

Evaluating Response Time, CPU and Memory across four locally hosted vision-language models on 600 logged requests routed through an n8n workflow.

Abstract

Multi-model workflows require a basis for deciding which model handles which request. This study benchmarked four locally hosted vision-language models on 600 logged requests routed through an n8n workflow, of which 460 completed and were analyzed, with each model contributing exactly 115 completed runs. Mean response time ranged from 11.8 s for Qwen to 71.8 s for Gemma, a sixfold gap; the 95% confidence intervals for the four means did not overlap, and bootstrap resampling reproduced the same ranking. Response times were right-skewed for every model (skewness 0.89 to 1.16), so a normal approximation misstated the share of Gemma calls completing within 60 s by 23 percentage points. Answer length correlated strongly with response time ($r = .91$ overall, $.82$ to $.98$ within models), while input type, trigger, and workflow instance showed no direct association. CPU percentage and memory did not differ detectably across models (Tukey adjusted $p \geq .89$), but estimated CPU-seconds did, because slower models held the processor longer: a typical Gemma call performed 5.07 times the processing work of a typical Qwen call. These results indicate that model choice, acting through how much a model writes and how quickly, is the principal lever on both latency and compute cost in this workflow.

Keywords: vision-language models, response time, skewed distributions, bootstrap resampling, workload routing

Choosing the Right Model for Each Task: Response Time, Compute Cost, and Routing Across Four Vision-Language Models

The Question We Set Out to Answer

This analysis addressed a single question: How does response time vary across models and request characteristics, and what patterns can guide model selection in a multi-model workflow? The sections that follow take the question in three parts, examining variation across models, variation across request characteristics, and the resource cost attached to each model.

The Answer in Brief

Table 1

Summary of Findings by Component of the Research Question

Component of the question	What the data shows
Across models	Average response time ranges from 11.8 s (Qwen) to 71.8 s (Gemma), a sixfold gap. Confidence intervals and bootstrap estimates both support this ranking, and the slower models also vary more from call to call.
Across request characteristics	Answer length is strongly associated with response time ($r = .91$ across 460 completed calls; .82 to .98 within each model). The structure search found no direct link from input type, trigger, or workflow instance to response time.
Patterns for model selection	Qwen is the leading candidate for short, time-sensitive tasks and MiniCPM for longer answers, subject to answer-quality testing. The expected length of an answer is a promising input for routing rules.

Key Numbers

- **Sixfold gap in average response time.** 11.8 s for the fastest model (Qwen) against 71.8 s for the slowest (Gemma).
- **$r = .91$** correlation between answer length and response time, the strongest request-level association found.

- **About twice as fast per 1,000 characters of answer.** MiniCPM and Qwen against Pixtral and Gemma.
- **5.07 times the estimated processing work** (CPU-seconds) for a typical Gemma call compared with a typical Qwen call.
- **19.7 CPU-hours** estimated saving for every 1,000 calls moved from Gemma to Qwen.
- **33 s versus 246 s** longest observed call for Qwen against Gemma.

What We Tested

Four locally hosted vision-language models were benchmarked on 600 logged requests sent through an n8n workflow running on two bot instances; the 460 requests that completed were analyzed. Each request was logged with its response time, average CPU and memory during the call, total workflow time, the bot instance that ran it, and the text the model returned. The test was balanced: every model completed exactly 115 runs and received the same mix of input types (text only, image only, text plus image, and mixed).

Table 2

Vision-Language Models Benchmarked

Model	Developer	Size
qwen2-vl-7b-instruct	Alibaba Qwen team	7B parameters
minicpm-o-2_6	OpenBMB	8B parameters, multimodal
pixtral-12b	Mistral AI	12B decoder plus 400M vision encoder
gemma-3-12b-it	Google DeepMind	12B parameters, 128,000-token context

Reliability did not separate the models. Each model had 115 completed calls, 3 logged failures, and 32 requests that never executed. The logs do not record why calls failed or did not execute, but the identical counts mean completion rates cannot favor one model over another.

Results at a Glance

Table 3

Response Time, Answer Length, and Estimated Processing Work by Model

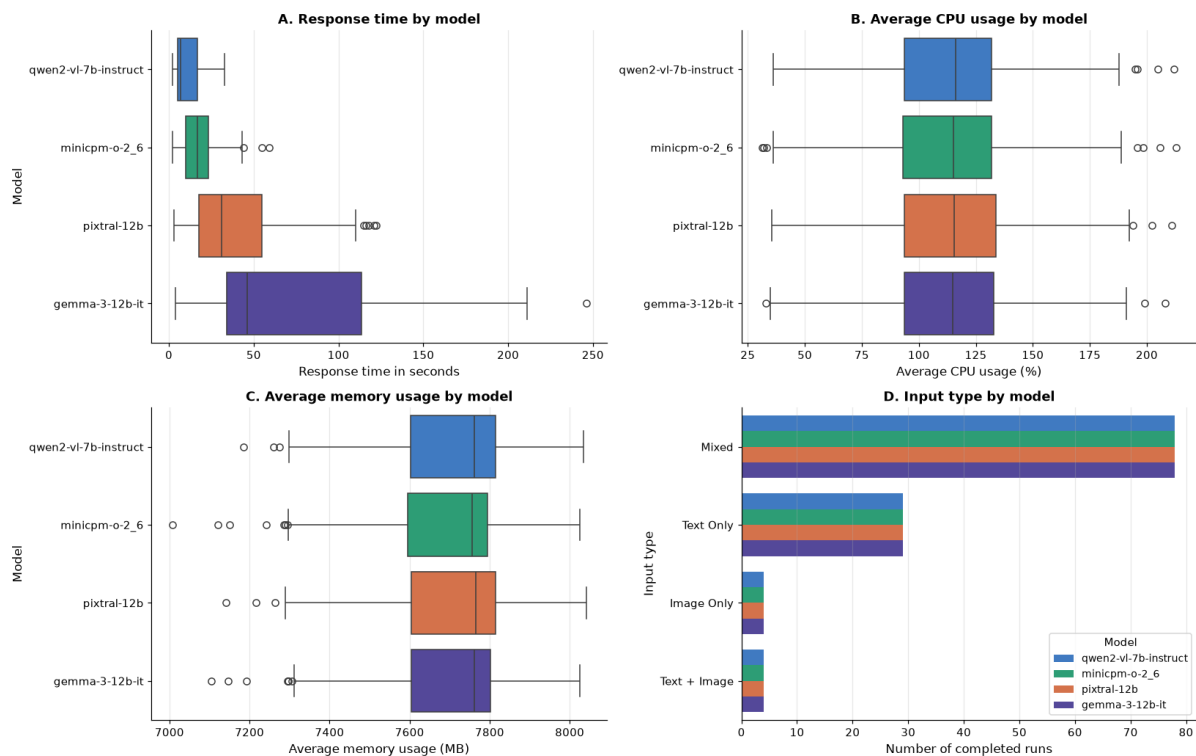
Model	Median call	Mean call	Longest call	Median answer length (characters)	Seconds per 1,000 characters	Median CPU-seconds
Qwen	7 s	11.8 s	33 s	471	15.0	9.8
MiniCPM	17 s	17.9 s	59 s	1,173	13.0	16.6
Pixtral	31 s	42.5 s	122 s	1,050	28.0	34.9
Gemma	46 s	71.8 s	246 s	1,194	32.8	49.9

Note. Models are listed fastest to slowest. Every value is drawn from the 460 completed runs, 115 per model. CPU-seconds are estimated as CPU percentage divided by 100, multiplied by call duration.

Part 1: Response Time Across Models

Finding 1: The Models Differ Sharply in Speed

Average response time ranged from 11.8 s (Qwen) to 71.8 s (Gemma). The spread grew with the average: the standard deviation was 8.2 s for Qwen and 56.7 s for Gemma. Gemma's middle half of calls alone spanned 34 to 113 s, which is wider than Qwen's entire range.

Figure 1*Response Time, CPU Usage, Memory Usage, and Input Type by Model*

Note. Each box covers the middle half of a model's calls, the line inside it is the median, and the dots are unusually slow or fast calls. In Panel A the boxes barely overlap and stretch further right as the models get slower, which is the speed gap in visual form. Panels B and C tell the opposite story: CPU and memory boxes sit at the same place for every model. Panel D confirms each model received the same mix of input types, so input type cannot explain the speed differences.

Finding 2: The Ranking Is Supported by Uncertainty Estimates

A standard error measures how precisely each average has been estimated. The 95% confidence intervals for the average did not overlap for any pair of models, and bootstrap estimates, which make no assumption about the shape of the data, gave the same ranking.

Table 4*Confidence Intervals for Mean and Median Response Time by Model*

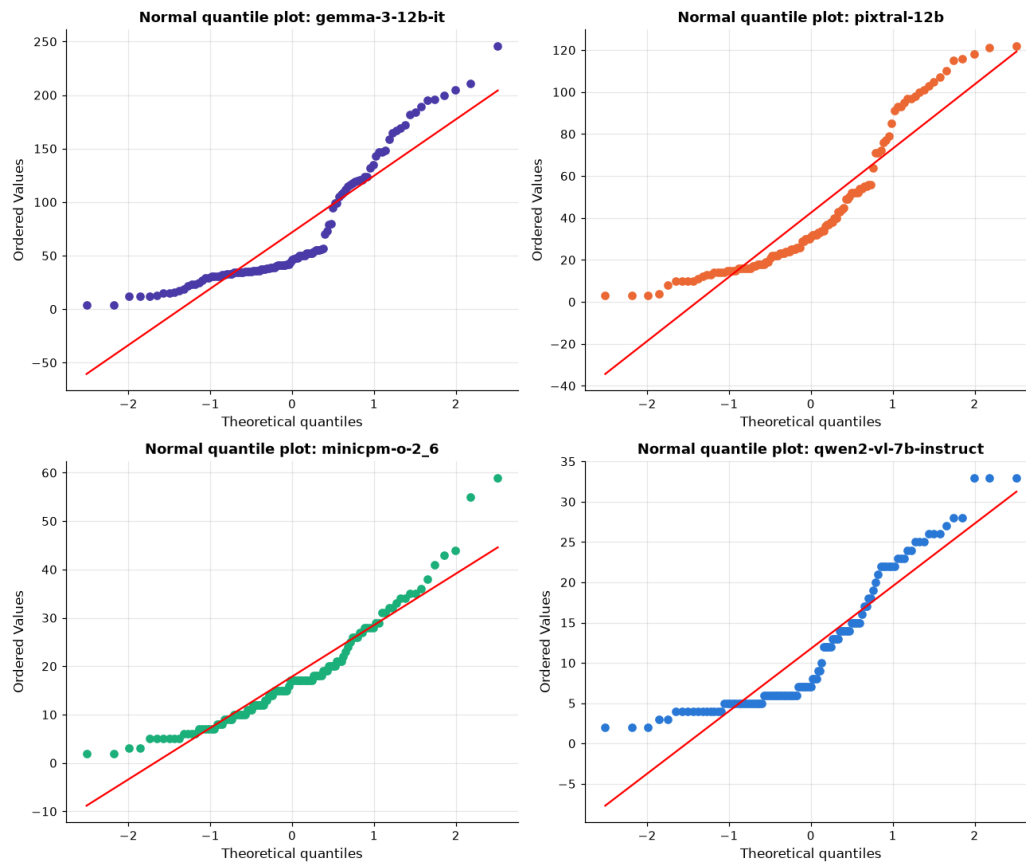
Model	95% CI for the mean call	95% CI for the median call
Qwen	[10.3, 13.3] s	[6, 12] s
MiniCPM	[15.9, 19.9] s	[15, 17] s
Pixtral	[36.5, 48.4] s	[25, 37] s
Gemma	[61.4, 82.3] s	[39, 52] s

Note. Intervals for the mean use the t distribution; intervals for the median use bootstrap resampling.

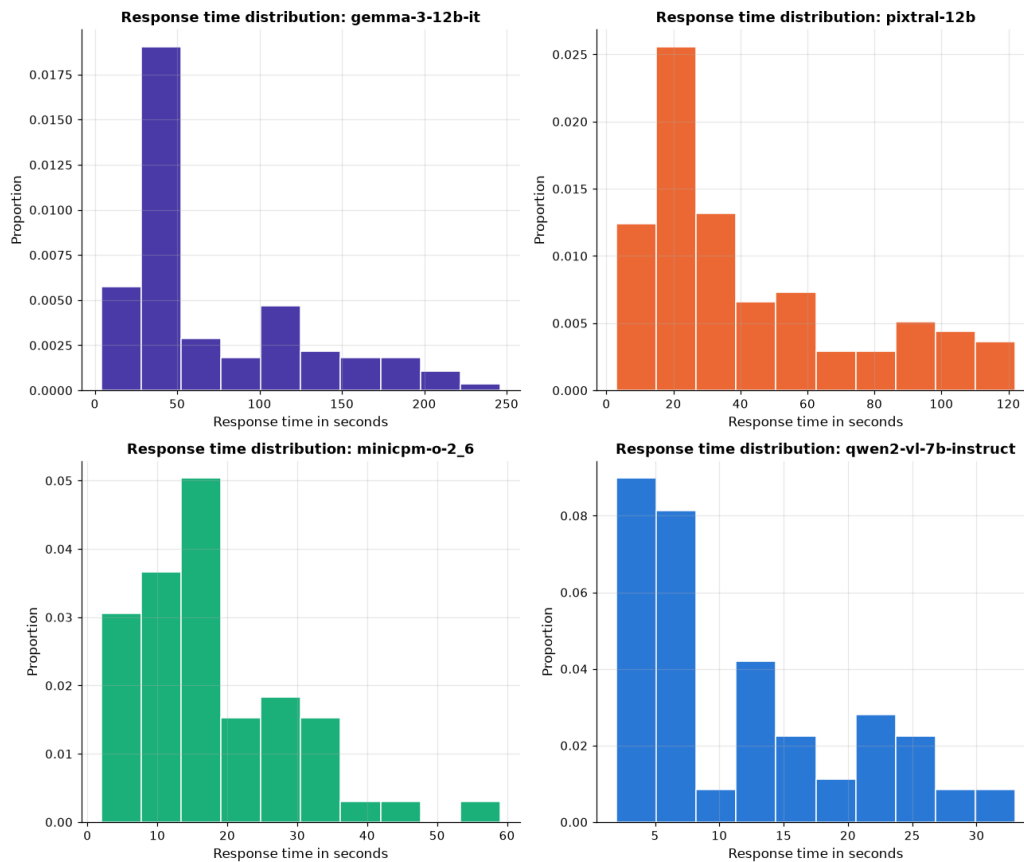
Bootstrap intervals for the mean agree with the t intervals within a few tenths of a second.

Finding 3: Plan for the Slow Tail, Not the Average

Response times were skewed to the right for every model (skewness 0.89 to 1.16): most calls were quick, with a minority of much slower ones. Three displays show this from different angles.

Figure 2*Normal Quantile Plots of Response Time by Model*

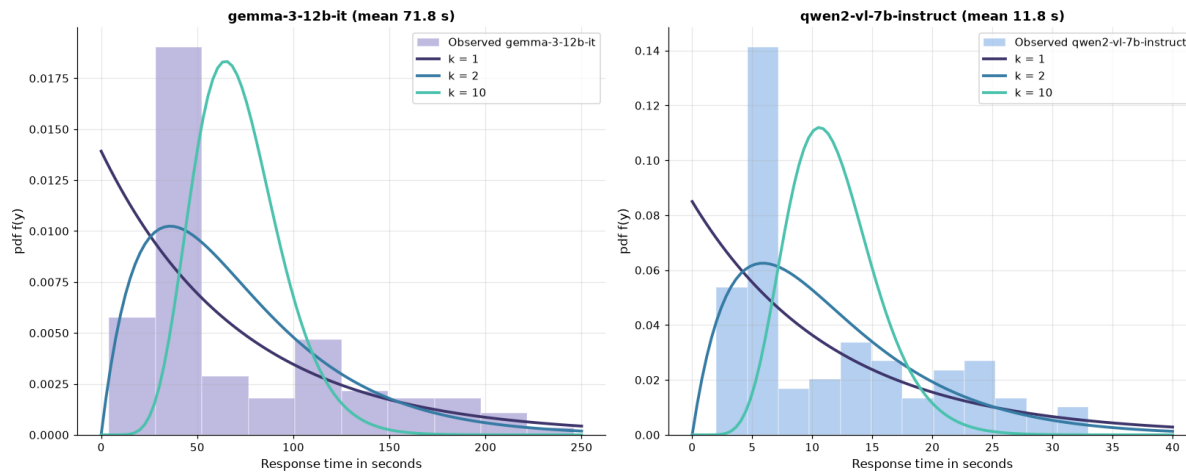
Note. If response times followed a normal distribution, the points would sit on the red line. Instead, every model's points bend upward at the top right, which means the slowest calls are much slower than a normal distribution would predict. The same bend appears in all four models, so one approach to planning covers all of them.

Figure 3*Distribution of Response Times by Model*

Note. Each bar shows the share of calls that fell in a time range. Every histogram peaks on the left and trails off to the right: most calls finish quickly, and a smaller group takes much longer. The time scales differ by panel; Qwen's axis ends at about 33 s, while Gemma's runs to 250 s.

Figure 4

Observed Response Times Against Three Gamma Curves for the Slowest and Fastest Models



Note. Curves share the same mean as the observed data. A Gamma curve is a standard shape for waiting times, and its shape value (k) sets how lopsided it is. For both Gemma and Qwen the observed bars follow the strongly lopsided $k = 2$ curve far better than the near-symmetric $k = 10$ curve. This supports treating response times as skewed when planning; it is a descriptive approximation, not a proven model.

That skew has two practical consequences:

- **For user expectations, quote the median.** A typical Gemma call takes 46 s, not the 72-s average.
- **For capacity and timeout planning, use the mean and the upper tail, not the mean alone.**

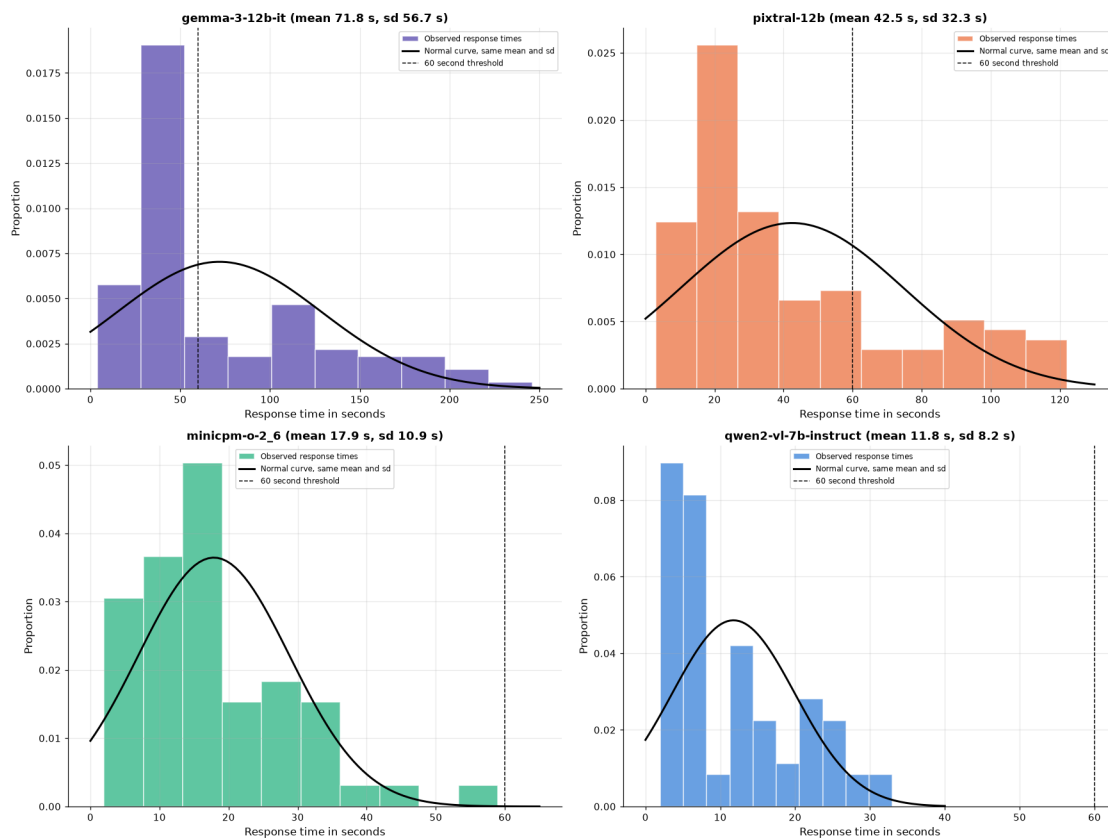
Totals across many calls follow the mean, and a timeout set at the mean would cut off a large share of legitimate calls. Future tails may also exceed the longest times seen in this benchmark.

A normal approximation gives the wrong answer for the slower models. Asked what share of calls finish within one minute, it misjudges Gemma by 23 percentage points.

Table 5*Predicted and Observed Share of Calls Completing Within 60 Seconds*

Model	Predicted by a normal curve	Observed
Qwen	100%	100%
MiniCPM	100%	100%
Pixtral	71%	77%
Gemma	42%	65%

Note. The 60-s threshold was chosen for this comparison; it is not a recorded service target.

Figure 5*Observed Response Times Against a Normal Curve With a 60-Second Threshold*

Note. The black curve is what a normal approximation expects; the bars are what actually happened, and the dashed line marks 60 s. For Gemma and Pixtral the curve is too flat: it misses the tall peak of

quick calls and spreads its weight across times where few calls fell, which is why it underestimates how many calls finish within a minute. For Qwen and MiniCPM the 60-s line sits beyond every observed call.

Part 2: Response Time Across Request Characteristics

Finding 4: Answer Length Is Strongly Associated With Response Time

A model writes its answer piece by piece, so a longer answer would be expected to take longer. Across 460 completed calls, answer length (in characters) and response time correlated at $r = .91$, where 1.0 would be a perfect straight-line match. The association held inside every model, so it is not merely a side effect of comparing a fast model with a slow one.

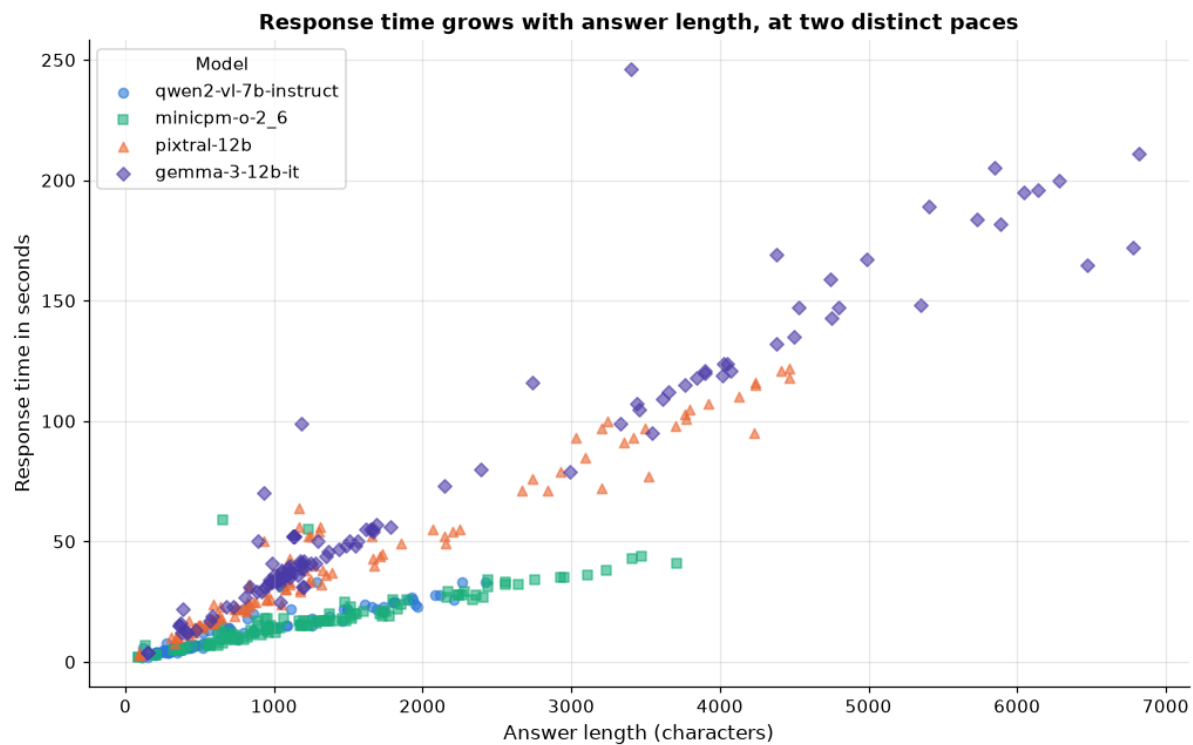
Table 6

Correlation Between Answer Length and Response Time by Model

Model	Correlation with response time	Median answer length (characters)
Qwen	$r = .95$	471
MiniCPM	$r = .82$	1,173
Pixtral	$r = .98$	1,050
Gemma	$r = .95$	1,194
All runs combined	$r = .91$	—

The models fall into two pace tiers. Measured as response seconds per 1,000 characters of answer, MiniCPM (13.0 s) and Qwen (15.0 s) take about half as long as Pixtral (28.0 s) and Gemma (32.8 s). This measure uses the full call time, so it is a practical pace figure rather than pure text-generation speed.

This helps explain the speed ranking. Qwen has the lowest response times and also writes the shortest answers. MiniCPM writes answers about as long as Gemma's in well under half the time per character, which makes it the stronger candidate when a task needs a long answer.

Figure 6*Response Time Against Answer Length by Model*

Note. Each point is one call. Points climb from bottom left to top right, so longer answers came with longer waits. They also form two separate bands: a steep band for Pixtral and Gemma (more seconds per character) and a shallow band for MiniCPM and Qwen. Correlation shows that the two measures move together; it does not on its own establish that length is the only cause.

These results carry an implication for routing. The actual answer length is known only after a call, but the expected length of an answer, such as a one-word classification against a full image description, is known in advance. That makes expected length a promising routing input, to be tested before it is relied on.

Finding 5: Request Setup Showed No Direct Link to Response Time

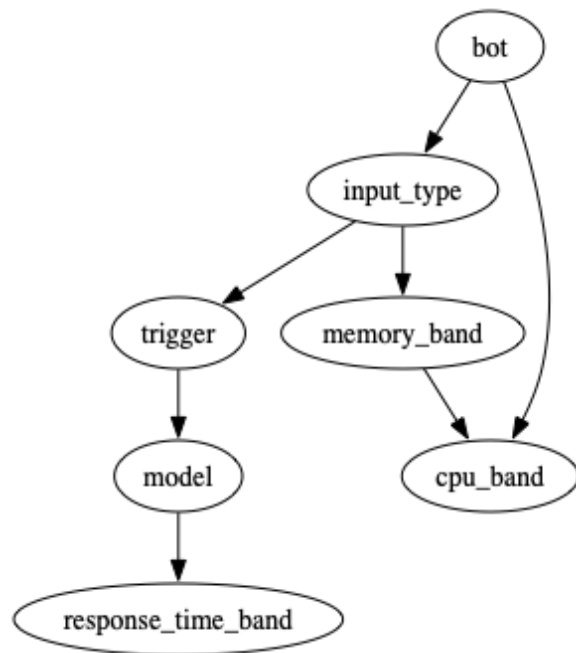
The other request characteristics in the log showed no direct link to response time:

- **Input type.** Every model received the same mix (Figure 1, Panel D), and the structure search described below selected no direct link from input type to response time.
- **Workflow instance.** Median response time was 22.0 s on Bot 1 and 23.5 s on Bot 2. The bots handled uneven numbers of calls (396 completed for Bot 1 and 64 for Bot 2), so similar medians suggest, rather than prove, equal speed.
- **Trigger.** No direct link was selected. Smaller or conditional effects have not been ruled out.

Finding 6: One Picture of What Drives What

The earlier findings tested variables one pair at a time. As a final step, a structure search examined every setup variable and every measured outcome together and selected the dependencies best supported by the data. The search, a score-based hill-climbing method using the Bayesian information criterion, tries many possible diagrams and keeps the simplest one that fits the 460 completed runs well. Response time, CPU, and memory were grouped into quartiles for this step.

One boundary was imposed on the search: it was not allowed to draw an arrow from a measured outcome (response time, CPU, memory) back to a setup choice (model, input type, trigger, bot), because something measured after a call cannot shape how that call was set up.

Figure 7*Learned Dependency Structure*

Note. This structure is exploratory. Each arrow is a dependency the search found worth keeping. The arrows are statistical links, not established causal relationships, and a missing arrow means no direct link was found, not that none exists.

- **Model → response time is the only arrow into response time.** Once the model is known, input type, trigger, and bot add no direct information. This points to model choice as the main lever.
- **No arrow runs from model to CPU or memory,** which agrees with the separate statistical tests in Finding 7.
- **Bot → CPU and input type → memory** link resource readings to the workflow rather than to the model (Finding 8).
- **Bot → input type → trigger → model** reflects how the benchmark scenarios were set up, not performance.

The diagram covers settings fixed before a call. Answer length is produced during the call, so it was analyzed separately in Finding 4. Read together, the evidence points to the model, acting through how much it writes and how fast, as the main factor in the wait.

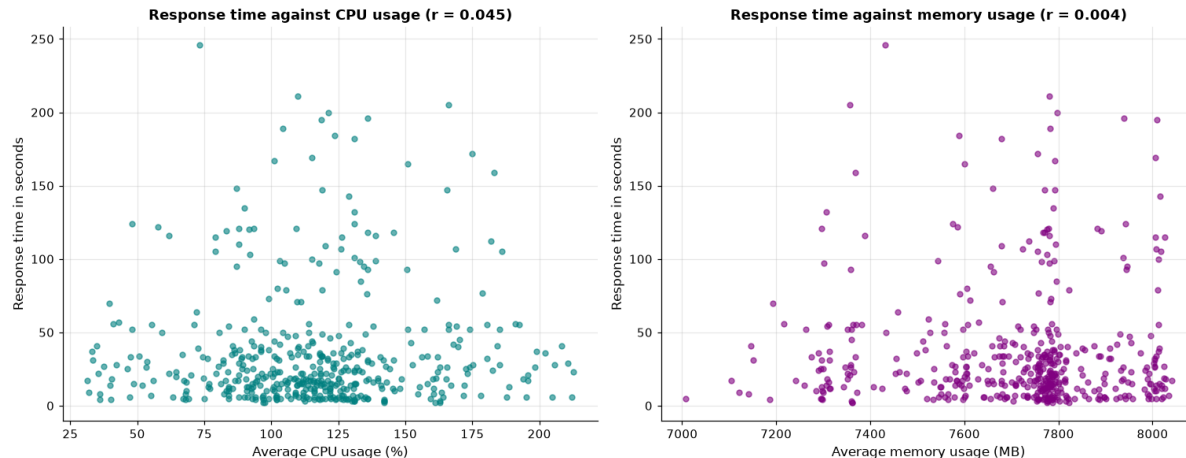
Part 3: The Resource Cost of Each Model

Finding 7: The Cost Difference Lies in How Long a Model Holds the Processor

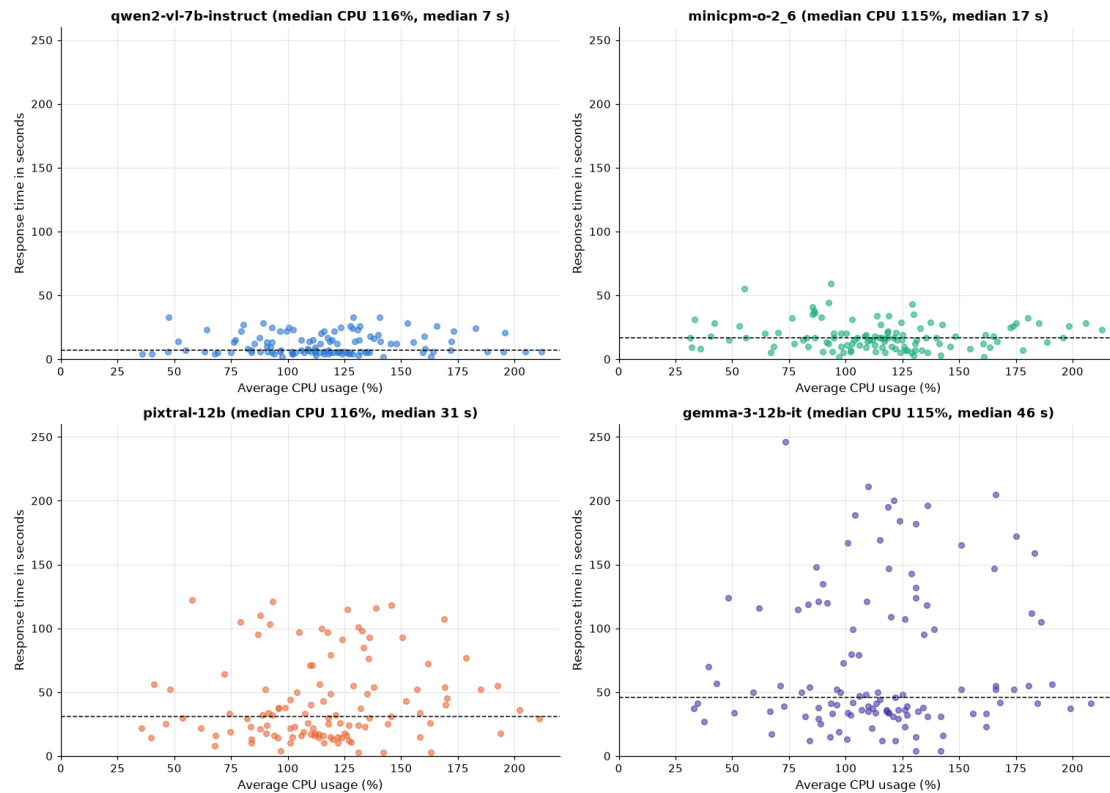
CPU use (around 115%, where 100% is one full core) and memory (around 7,700 MB) showed no statistically significant differences between models (Tukey pairwise tests, adjusted $p \geq .89$), and neither moved with response time ($r = .05$ for CPU, $r = .00$ for memory). Taken alone, those readings would suggest that model choice carries no resource cost.

Figure 8

Response Time Against CPU Use and Memory Usage



Note. All completed calls are plotted; CPU use appears on the left and memory on the right. If slow calls drew more CPU or memory, the points would rise from left to right. Instead both panels show a shapeless cloud.

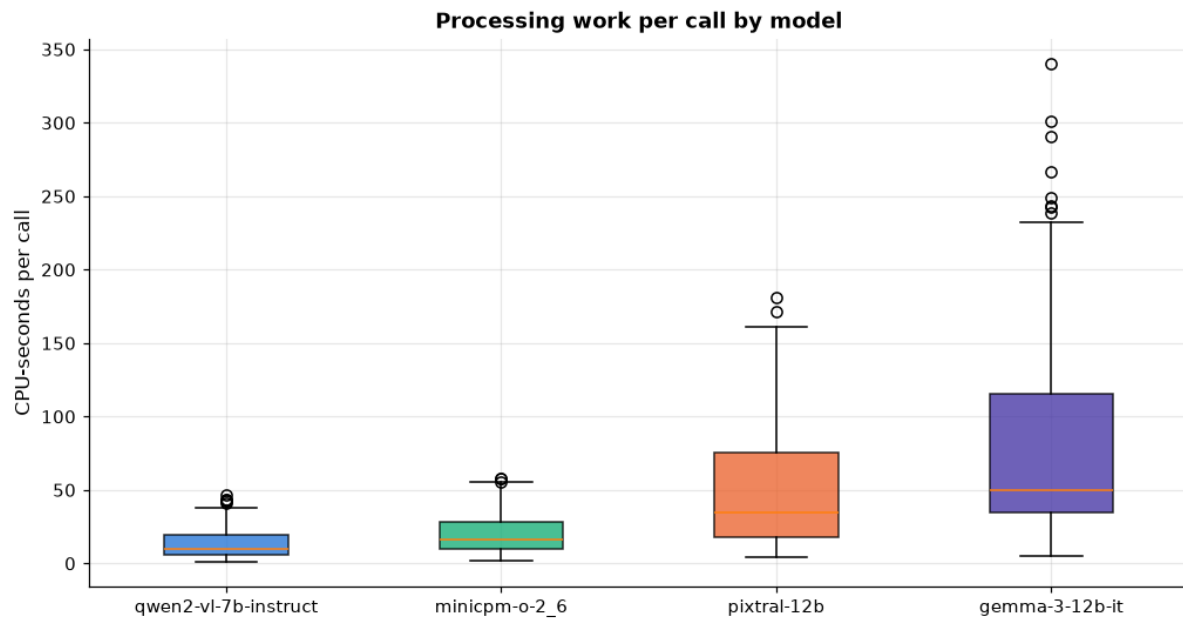
Figure 9*CPU Use Against Response Time by Model*

Note. All four panels share the same scales, and the dashed line marks each model's median response time. Every model spreads across the same CPU range horizontally, but the models reach very different heights: Qwen stays below 35 s while Gemma reaches 246 s. The models run the processor at the same intensity; the slower ones simply run it for longer.

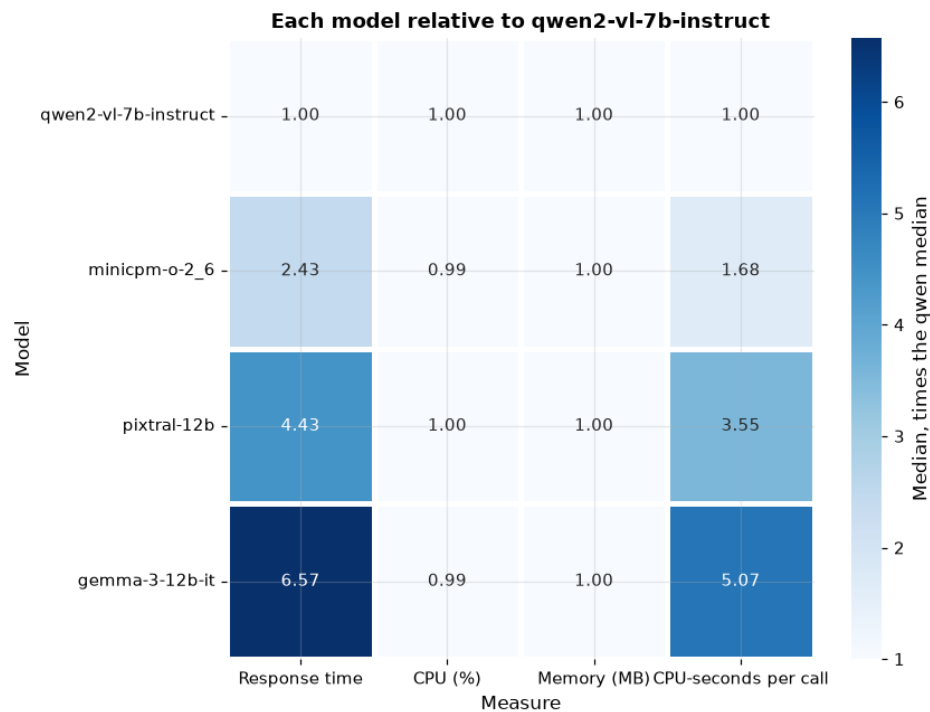
That duration is where the cost lies. Multiplying CPU use by call duration gives estimated CPU-seconds, an approximation of the processing work in each call (CPU percentage divided by 100, multiplied by seconds). It is a planning estimate, not a direct monetary or energy measurement. On this measure the models separate clearly.

Figure 10

Estimated CPU-Seconds per Call by Model



Note. Unlike the flat CPU-percentage boxes in Figure 1, these boxes climb steadily from Qwen to Gemma. A typical Gemma call performs about five times the processing work of a typical Qwen call, and its long upper whisker shows that some Gemma calls cost far more.

Figure 11*Median Values Relative to Qwen by Measure and Model*

Note. A value of 1.00 equals Qwen. Darker cells indicate a larger multiple of Qwen. The CPU and memory columns stay at 1.00 for every model, while the response time and CPU-seconds columns climb to 6.57 and 5.07 for Gemma. This is the clearest single view of the finding: the same intensity, with very different duration and total work.

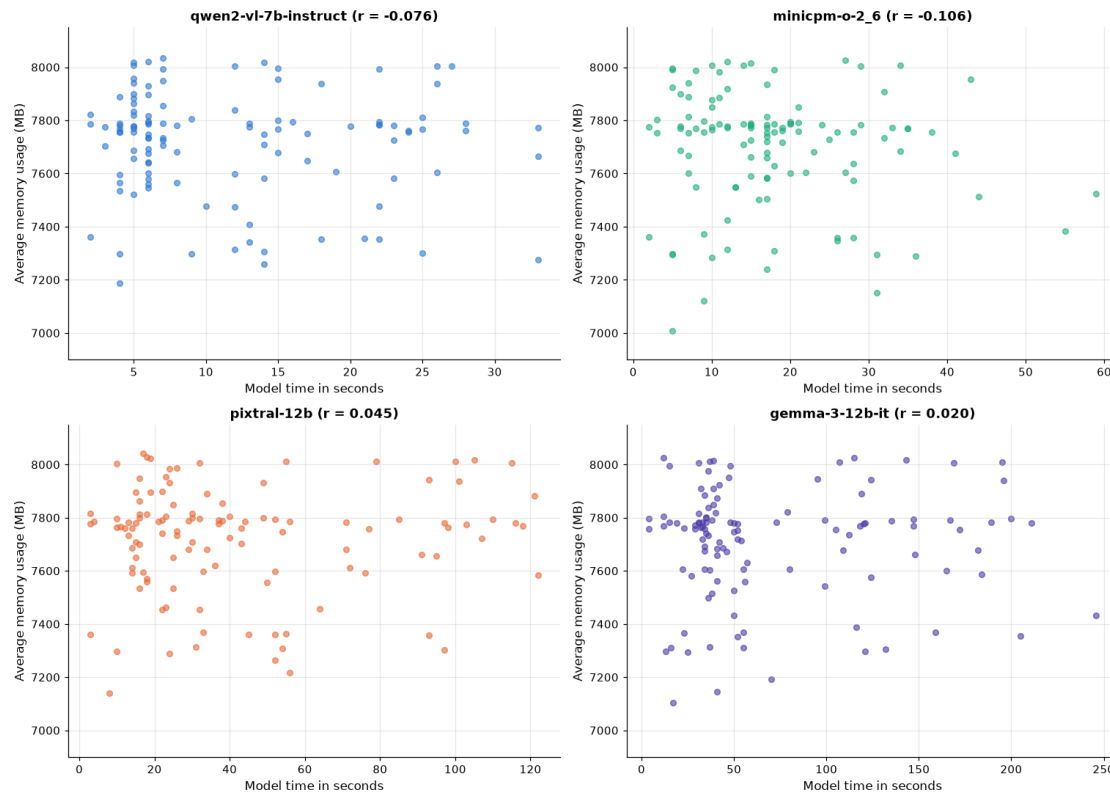
Table 7*Estimated Processing Savings From Routing Calls to Qwen*

Model replaced by Qwen	Median CPU-seconds relative to Qwen	Estimated CPU-hours saved per 1,000 calls
MiniCPM	1.68	1.8
Pixtral	3.55	10.1
Gemma	5.07	19.7

Note. Savings use mean CPU-seconds, since totals across many calls follow the mean. The estimates assume the substitute model could handle the same work at acceptable quality, which this benchmark did not test.

Finding 8: Memory and CPU Load Track the Workflow, Not the Model

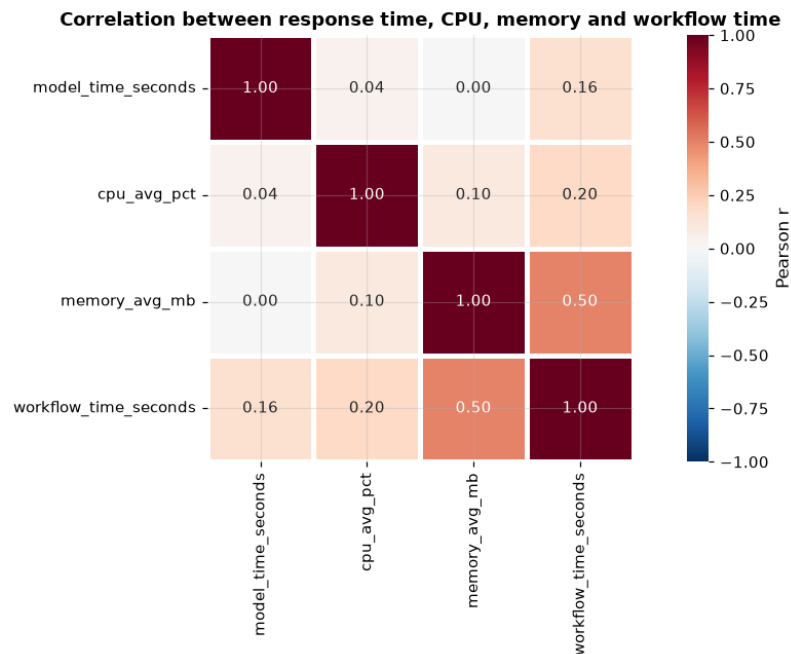
Memory had a moderate correlation with total workflow duration ($r = .50$) but almost none with the model call itself ($r = .00$), and the pattern was nearly identical for all four models. Model calls are short relative to the whole workflow: the median model call was 7.6% of the median workflow time.

Figure 12*Memory Usage Against Response Time by Model*

Note. Each panel shows a flat cloud with a correlation near zero, ranging from $-.11$ to $.05$. However long a model takes, memory stays in the same range, so choosing a different model would not be expected to change memory needs.

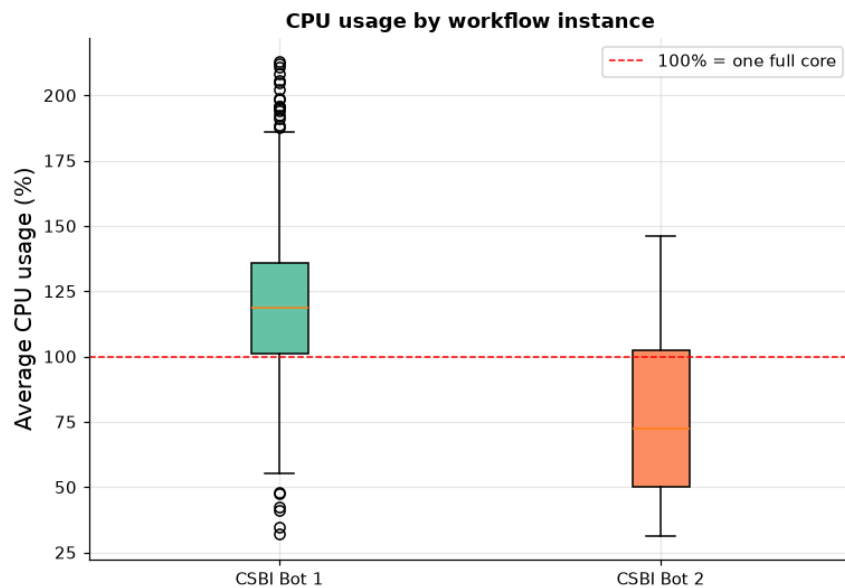
Figure 13

Correlations Among Response Time, CPU Use, Memory, and Workflow Time



Note. Each cell shows how strongly two measures move together, on a scale from -1 to 1 . The only notable link is memory with workflow time (.50). Response time is almost unrelated to CPU (.04) and memory (.00), and only weakly related to total workflow time (.16).

CPU load, by contrast, differed by workflow instance.

Figure 14*CPU Use by Workflow Instance*

Note. The red line marks one full core. Bot 1 ran at a median of 119% CPU, above one full core, while Bot 2 ran at 72%, even though both bots showed nearly the same model response times. Bot 2 contributed only 64 completed calls, so this is a strong signal worth confirming with more data.

Patterns for Model Selection

Table 8*Summary of Findings and Supporting Evidence*

Finding	Evidence
Models differ in speed	Means 11.8 to 71.8 s; all 95% confidence intervals separate
Response times are right-skewed	Skewness 0.89 to 1.16; mean above median for every model
Slower models vary more	<i>SD</i> 8.2 s (Qwen) to 56.7 s (Gemma); longest Gemma call 246 s
Answer length is associated with response time	$r = .91$ overall; two distinct pace tiers
No direct setup-to-response-time links found	No edge from input type, trigger, or bot to response time
CPU percentage and memory differences not detected	Tukey adjusted $p \geq .89$; r near zero with response time
Estimated CPU-seconds vary by model	Gemma 5.07 times Qwen; about 19.7 CPU-hours per 1,000 calls
Memory relates to the workflow	r about .50 with workflow time, near zero with model time

Recommendations**Routing****Table 9***Recommended Model by Task Type*

Task type	Candidate model	Rationale
Short, time-sensitive (classification, tagging, quick replies)	Qwen	Lowest median and mean response time; longest observed call 33 s; lowest estimated compute. Confirm answer quality before adopting.
Longer answers (image descriptions, summaries)	MiniCPM	Lowest response seconds per 1,000 characters; median and mean within 1 s of each other. Validate with matched tasks and quality scoring.
Tasks with specialist quality needs	Pixtral or Gemma	This benchmark measures speed, not accuracy. Use task-based quality tests to justify two to five times the wait and compute.

Appendix

How the Analysis Was Done

All results come from the 460 completed runs (115 per model). The following methods were used:

- Descriptive statistics and box plots per model for response time, CPU, and memory.
- Distribution checks (normal quantile plots, histograms, skewness, and Gamma shape comparison) showing right-skewed response times. Gamma is used as a descriptive approximation, not a proven fit.
- Standard errors and 95% confidence intervals for mean response time using the t distribution and bootstrap resampling, plus bootstrap intervals for the median.
- Tukey pairwise comparisons for CPU and memory between models.
- Correlation analysis, estimated CPU-seconds (CPU percentage divided by 100, multiplied by seconds), and response seconds per 1,000 characters of answer.
- An exploratory score-based dependency structure search (Bayesian information criterion, hill climbing) across setup variables and banded outcome variables.